Green University of Bangladesh

Department of Computer Science and Engineering (CSE)

Faculty of Sciences and Engineering

Semester: (Spring, Year:2026), B.Sc. in CSE (Day)

Lab Report NO: 02

Course Title: Artificial Intelligence Lab

Course Code: CSE 316 **Section:** 232_D7

Lab Experiment Name: Heuristic A* Search Algorithm

Student Details

Name	ID
MD:AL-Mamun	232002264

Lab Date : 15-03-2026

Submission Date : 28-03-2026

Course Teacher's Name : Md. Sabbir Hosen Mamun

Lab Report Status

Marks: …… **Signature:.....**

Comments:..... **Date:.....**

1. TITLE OF THE LAB REPORT EXPERIMENT

Implementation of A* Search Algorithm for Path Finding in a 2D Grid Maze

2. OBJECTIVES/AIM

- Implement the A* search algorithm to determine the shortest route in a 2D grid-based maze.
- Utilize the Manhattan distance heuristic to efficiently direct the search toward the goal cell.
- Evaluate how the A* algorithm performs when obstacles are present in the maze.
- Create a Python program that can read maze input from a file and output the optimal path along with its total traversal cost.

3. PROCEDURE / ANALYSIS / DESIGN

The A* search algorithm is implemented step-by-step as follows:

Step 1: Input Reading

- Extract the number of rows (R) and columns (C) from the input file.
- Load the grid data, where `0` represents an open cell and `1` represents an obstacle.
- Identify the coordinates of the starting cell `(sr, sc)` and the goal cell `(tr, tc)` from the file

Step 2: Grid to Graph Conversion

- Cells with a value of `0` are considered as nodes in the grid.
- Each node is linked to its adjacent valid neighbors in the four directions: up, down, left, and right

Step 3: Heuristic Calculation

- Compute Manhattan Distance for each

  node: $H(n) = |x1 - x2| + |y1 - y2|$

Step 4: Initialization

- Initialize a priority queue (min-heap).
- Insert the start node with cost:

  $G = 0$, H = heuristic value, $F = G + H$

- Create:
  - Visited dictionary (to store best cost)
  - Parent dictionary (to reconstruct path)

Step 5: A Search Execution*

- Extract node with minimum F value from priority queue.
- If target is reached → stop search.
- Otherwise, explore all valid neighbors.

Step 6: Cost Update

- For each neighbor:
  - Calculate new G cost.
  - If this path is better, update:
    - Visited cost
    - Parent pointer
    - Push into priority queue

Step 7: Path Reconstruction

- After reaching target, backtrack using parent dictionary.
- Construct the shortest path from start to goal.

4. IMPLEMENTATION

Code:

```
import heapq


def manhattan_distance(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])


def a_star(grid, start, goal):
    rows, cols = len(grid), len(grid[0])

    # Priority Queue (f, node)
    open_set = []
    heapq.heappush(open_set, (0, start))

    # g(n) cost
    g_cost = {start: 0}

    # parent for path
    parent = {}

    # visited (closed set)
    closed_set = set()

    while open_set:
        current_f, current = heapq.heappop(open_set)
```

```python
        if current == goal:
            path = []
            while current in parent:
                path.append(current)
                current = parent[current]
            path.append(start)
            path.reverse()
            return path

        if current in closed_set:
            continue
        closed_set.add(current)

        r, c = current

        directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

        for dr, dc in directions:
            nr, nc = r + dr, c + dc

            if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 0:
                neighbor = (nr, nc)

                new_g = g_cost[current] + 1

                if neighbor not in g_cost or new_g < g_cost[neighbor]:
                    g_cost[neighbor] = new_g

                    f = new_g + manhattan_distance(neighbor, goal)
                    heapq.heappush(open_set, (f, neighbor))

                    parent[neighbor] = current

    return None


R, C = map(int, input("Enter rows and cols: ").split())

grid = []
print("Enter grid (0 = free, 1 = wall):")
for _ in range(R):
    grid.append(list(map(int, input().split())))

sr, sc = map(int, input("Start (row col): ").split())
tr, tc = map(int, input("Target (row col): ").split())

start = (sr, sc)
goal = (tr, tc)

path = a_star(grid, start, goal)

if path:
    print(" Shortest Path:", path)
    print("Cost:", len(path) - 1)
else:
    print("Path not found")
```

5. TEST RESULT / OUTPUT

Case 1:

```
PS C:\xampp\htdocs\python> & "C:\Users\SARK\AppData\Local\Programs\Python\Python314\python.exe" first.py
Enter rows and cols: 4 4
Enter grid (0 = free, 1 = wall):
0 0 0 0
1 1 0 1
0 0 0 0
0 1 1 0
Start (row col): 0 0
Target (row col): 3 3
 Shortest Path: [(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (3, 3)]
Cost: 6
```

Case 2:

```
PS C:\xampp\htdocs\python> & "C:\Users\SARK\AppData\Local\Programs\Python\Python314\python.exe" first.py
Enter rows and cols: 3 3
Enter grid (0 = free, 1 = wall):
0 1 0
0 1 0
0 1 0
Start (row col): 0 0
Target (row col): 2 2
Path not found
PS C:\xampp\htdocs\python>
```

6. ANALYSIS AND DISCUSSION

The A* search algorithm is an informed search strategy that efficiently identifies the shortest path by combining the actual path cost (G) with a heuristic estimate (H). In this study, the Manhattan distance heuristic is employed since movement is restricted to four directions. This heuristic guides the algorithm toward the target cell, minimizing unnecessary exploration of the grid. Consequently, A* examines fewer nodes than uninformed search methods such as BFS or DFS, while still guaranteeing an optimal path when one exists.

Regarding performance, A* is effective for grids of moderate size, though in highly obstructed or very large grids, it may still process a significant number of nodes. The use of a priority queue ensures that nodes with the lowest estimated total cost are always expanded first, enhancing efficiency. Despite this, time complexity can increase due to repeated evaluations of nodes. Overall, A* remains a reliable and practical approach for solving shortest path problems in grid-based mazes.

7. SUMMARY

In this lab, the A* search algorithm is implemented to determine the shortest path in a 2D grid maze with obstacles. The Manhattan distance heuristic is applied to guide the search efficiently toward the goal. By prioritizing nodes likely to lead to the optimal path, the algorithm examines fewer cells while still guaranteeing the shortest route when it exists. The program reads input from a file and outputs either the optimal path along with its total cost or indicates failure if no path is available.

For source code and documentation are given below

- **GitHub Link:**